

CHAROTAR UNIVERSITY OF SCIENCE & TECHNOLOGY

Faculty of Technology & Engineering

Devang Patel Institute of Advance Technology and Research (DEPSTAR)

CSUE301 – : Big Data Analytics

Practical File – ODD Semester 2026-27

StudentID: D25DIT081

Student Name: Rudra Maiyariya

Practical 6

1. Problem Definition

Scenario:

ShopSmart Analytics Pvt. Ltd. wants to analyze its e-commerce sales data. The company needs to identify high-performing product categories, top-selling products and revenue contribution by city. Since the amount of data is large, Apache Spark is used for scalable data processing and analysis.

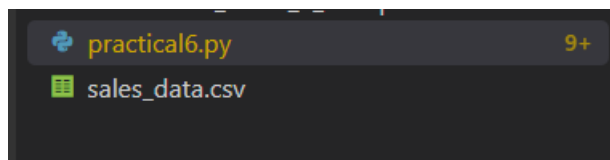
Objective

To use PySpark DataFrames to:

- Load e-commerce sales data
- Validate the dataset
- Calculate revenue
- Find category-wise revenue
- Find top-selling products
- Calculate city-wise revenue
- Sort categories according to revenue
- Generate analytical summaries
- Extract business insights

PART A — Practical Execution

Step 1: Creating Folder and files



practical6.py sales_data.csv X

sales_data.csv

	Product_ID	Sale_Date	Sales_Rep	Region	Sales_Amount	Quantity_Sold	Product_Category	Unit_Cost	Unit_Price	Customer_Type	Discount	Pay
1	1052	2023-02-03	Bob	North	5053.97	18	Furniture	152.75	267.22	Returning	0.09	Cash,Online,North-Bob
2	1093	2023-04-21	Bob	West	4384.02	17	Furniture	3816.39	4209.44	Returning	0.11	Cash,Retail,West-Bob
3	1015	2023-09-21	David	South	4631.23	30	Food	261.56	371.4	Returning	0.2	Bank Transfer,Retail,South-David
4	1072	2023-08-24	Bob	South	2167.94	39	Clothing	4330.03	4467.75	New	0.02	Credit Card,Retail,South-Bob
5	1061	2023-03-24	Charlie	East	3750.2	13	Electronics	637.37	692.71	New	0.08	Credit Card,Online,East-Charlie
6	1021	2023-02-11	Charlie	West	3761.15	32	Food	900.79	1106.51	New	0.21	Cash,Online,West-Charlie
7	1083	2023-04-11	Bob	West	618.31	29	Furniture	2408.81	2624.09	Returning	0.14	Cash,Online,West-Bob
8	1087	2023-01-06	Eve	South	7698.92	46	Furniture	3702.51	3964.65	New	0.12	Bank Transfer,Online,South-Eve
9	1075	2023-06-29	David	South	4223.39	30	Furniture	738.06	1095.4499999999998	New	0.05	Bank Transfer,Online,South-David
10	1075	2023-10-09	Charlie	West	8239.58	18	Clothing	2228.35	2682.34	New	0.13	Bank Transfer,Online,West-Charlie
11	1088	2023-11-16	Eve	North	8518.45	13	Furniture	2440.11	2517.6	New	0.23	Bank Transfer,Retail,North-Eve
12	1100	2023-08-14	Bob	West	2198.74	43	Food	1100.81	1137.44	Returning	0.08	Bank Transfer,Online,West-Bob
13	1024	2023-11-11	Eve	West	6607.8	21	Food	622.01	641.09	Returning	0.0	Bank Transfer,Retail,West-Eve
14	1003	2023-12-31	Alice	South	4775.59	30	Furniture	4190.28	4270.65	New	0.2	Cash,Online,South-Alice
15	1022	2023-08-17	Charlie	South	8813.55	21	Food	2537.2	2869.6	New	0.29	Bank Transfer,Retail,South-Charlie
16	1053	2023-10-16	Bob	North	2235.83	48	Furniture	121.19	487.65	New	0.18	Cash,Retail,North-Bob
17	1002	2023-05-30	David	North	6810.35	17	Furniture	4024.76	4420.1500000000001	Returning	0.04	Cash,Online,North-David
18	1088	2023-10-04	Bob	East	6116.75	40	Electronics	4904.93	5034.35	New	0.1	Credit Card,Retail,East-Bob
19	1030	2023-07-17	David	West	3023.48	19	Clothing	3049.33	3209.22	Returning	0.26	Cash,Online,West-David
20	1038	2023-03-11	Bob	South	1452.35	15	Clothing	2543.36	2790.1000000000004	Returning	0.07	Cash,Online,South-Bob
21	1002	2023-04-22	Eve	North	6551.23	9	Electronics	4398.16	4439.12	New	0.18	Bank Transfer,Online,North-Eve
22	1064	2023-01-04	Eve	East	7412.11	10	Electronics	4764.96	5074.42	New	0.12	Cash,Online,East-Eve
23	1060	2023-12-16	Eve	East	3224.71	44	Clothing	3784.96	4276.99	New	0.06	Bank Transfer,Retail,East-Eve
24												

2. Output of Practical6.py

```

-----
Starting e-commerce sales analysis using Spark DataFrame
-----
Sales data loaded
Displaying the first few records
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
|Product_ID|Sale_Date|Sales_Rep|Region|Sales_Amount|Quantity_Sold|Product_Category|Unit_Cost|Unit_Price|Customer_Type|Discount|Payment_Method|
|Sales_Channel|Region_and_Sales_Rep|
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
|1052|2023-02-03|Bob|North|5053.97|18|Furniture|152.75|267.22|Returning|0.09|Cash|
|Online|North-Bob|
|1093|2023-04-21|Bob|West|4384.02|17|Furniture|3816.39|4209.44|Returning|0.11|Cash|
|Retail|West-Bob|
|1015|2023-09-21|David|South|4631.23|30|Food|261.56|371.4|Returning|0.2|Bank Transfer|
|Retail|South-David|
|1072|2023-08-24|Bob|South|2167.94|39|Clothing|4330.03|4467.75|New|0.02|Credit Card|
|Retail|South-Bob|
|1061|2023-03-24|Charlie|East|3750.2|13|Electronics|637.37|692.71|New|0.08|Credit Card|
|Online|East-Charlie|
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+

```

```

+-----+
only showing top 5 rows
-----
Schema of the dataset
root
 |-- Product_ID: integer (nullable = true)
 |-- Sale_Date: date (nullable = true)
 |-- Sales_Rep: string (nullable = true)
 |-- Region: string (nullable = true)
 |-- Sales_Amount: double (nullable = true)
 |-- Quantity_Sold: integer (nullable = true)
 |-- Product_Category: string (nullable = true)
 |-- Unit_Cost: double (nullable = true)
 |-- Unit_Price: double (nullable = true)
 |-- Customer_Type: string (nullable = true)
 |-- Discount: double (nullable = true)
 |-- Payment_Method: string (nullable = true)
 |-- Sales_Channel: string (nullable = true)
 |-- Region_and_Sales_Rep: string (nullable = true)

```

```

-----
Column names in the dataset
['Product_ID', 'Sale_Date', 'Sales_Rep', 'Region', 'Sales_Amount', 'Quantity_Sold', 'Product_Category', 'Unit_Cost', 'Unit_Price', 'Customer_Type', 'Discount', 'Payment_Method', 'Sales_Channel', 'Region_and_Sales_Rep']
Number of rows: 1000
Number of columns: 14
-----
Checking data types
Product_ID : int
Sale_Date : date
Sales_Rep : string
Region : string
Sales_Amount : double
Quantity_Sold : int
Product_Category : string
Unit_Cost : double
Unit_Price : double
Customer_Type : string
Discount : double
Payment_Method : string
Sales_Channel : string
Region_and_Sales_Rep : string
-----

```

```

-----
Checking missing values
Product_ID : 0
Sale_Date : 0
Sales_Rep : 0
Region : 0
Sales_Amount : 0
Quantity_Sold : 0
Product_Category : 0
Unit_Cost : 0
Unit_Price : 0
Customer_Type : 0
Discount : 0
Payment_Method : 0
Sales_Channel : 0
Region_and_Sales_Rep : 0
-----

```

```

-----
Revenue column created using Quantity Sold multiplied by Unit Price
Displaying sales data with Revenue
+-----+-----+-----+-----+-----+
|Product_ID|Product_Category|Quantity_Sold|Unit_Price|Revenue|
+-----+-----+-----+-----+-----+
|1052|Furniture|18|267.22|4809.960000000001|
|1093|Furniture|17|4209.44|71560.48|
|1015|Food|30|371.4|11142.0|
|1072|Clothing|39|4467.75|174242.25|
|1061|Electronics|13|692.71|9005.23|
|1021|Food|32|1106.51|35408.32|
|1083|Furniture|29|2624.09|76098.61|
|1087|Furniture|46|3964.65|182373.9|
|1075|Furniture|30|1095.4499999999998|32863.49999999999|
|1075|Clothing|18|2682.34|48282.12|
+-----+-----+-----+-----+-----+
only showing top 10 rows
-----

```

```

-----
Category wise revenue
+-----+-----+
|Product_Category|Total_Revenue|
+-----+-----+
|Clothing|1.9286733929999996E7|
|Furniture|1.8330280560000006E7|
|Electronics|1.7571040570000004E7|
|Food|1.5141885649999999E7|
+-----+-----+
-----

```

```

-----
Top selling products based on quantity sold
+-----+-----+
|Product_ID|Total_Quantity_Sold|
+-----+-----+
|1090|590|
|1092|548|
|1062|506|
|1099|502|
|1058|452|
|1008|407|
|1033|404|
|1024|398|
|1061|391|
|1089|390|
+-----+-----+
only showing top 10 rows
-----

```

```
-----
Region wise revenue
```

```
+-----+
|Region|      Total_Revenue|
+-----+
| North|1.8208403850000013E7|
|  East| 1.807740105000001E7|
|  West| 1.776156864999999E7|
| South|1.6282567160000017E7|
+-----+
```

```
-----
Categories sorted according to revenue
```

```
+-----+
|Product_Category|      Total_Revenue|
+-----+
|      Clothing|1.9286733929999996E7|
|      Furniture|1.8330280560000006E7|
|      Electronics|1.7571040570000004E7|
|          Food|1.5141885649999999E7|
+-----+
```

```
-----
Total Revenue: 70329940.70999995
Total Quantity Sold: 25355
Average Revenue per Transaction: 70329.94070999995
Total Number of Orders: 1000
-----
```

```
Top 3 revenue generating regions
```

```
+-----+
|Region|      Total_Revenue|
+-----+
| North|1.8208403850000013E7|
|  East| 1.807740105000001E7|
|  West| 1.776156864999999E7|
+-----+
```

```
only showing top 3 rows
```

```
-----
Average revenue per order
```

```
+-----+
|Average_Revenue_Per_Order|
+-----+
|          70329.94070999995|
+-----+
```

```
-----
Least performing product category
```

```
+-----+
|Product_Category|      Total_Revenue|
+-----+
|          Food|1.5141885649999999E7|
+-----+
```

```
only showing top 1 row
```

```
-----
Final sales analysis completed
```

```
SUCCESS: The process with PID 28952 (child process of PID 20936) has been terminated.
SUCCESS: The process with PID 20936 (child process of PID 18324) has been terminated.
SUCCESS: The process with PID 18324 (child process of PID 22696) has been terminated.
```

3. Key Questions / Analysis

Q1. Why are Spark DataFrames preferred over traditional spreadsheets for large-scale analytics?

Spark DataFrames are preferred because they can process very large datasets efficiently using distributed computing. Data can be divided into partitions and processed in parallel across multiple machines. Spreadsheets are more suitable for small or moderate-sized datasets and have limitations when the data becomes very large.

Q2. What is the role of the `groupBy()` operation in data aggregation?

The `groupBy()` operation groups records having the same value in a particular column. It is commonly used with aggregation functions such as `sum()`, `avg()`, `count()`, `min()` and `max()`.

For example:

```
df.groupBy("Category").sum("Revenue")
```

groups the sales according to product category and calculates the total revenue for each category.

Q3. Explain the difference between transformations and actions in Spark.

Transformations create a new DataFrame or RDD from existing data. They are lazily evaluated.

Examples:

```
select()
```

```
filter()
```

```
withColumn()
```

```
groupBy()
```

Actions trigger the actual execution of Spark operations and return a result or output.

Examples:

```
show()
```

```
count()
```

```
collect()
```

Therefore:

Transformation → Defines the operation

Action → Executes the operation

Q4. How does Spark improve scalability for e-commerce analytics?

Spark improves scalability by using distributed and parallel processing. Large e-commerce datasets can be divided into partitions and processed simultaneously across multiple computing resources. Spark DataFrames also provide optimized execution, making large-scale sales analysis faster and more efficient.

Q5. Which category contributes the highest revenue and what business action can be taken?

The highest-revenue category is identified from the category-wise revenue analysis by sorting categories in descending order of revenue.

Highest Revenue Category: Clothing

A suitable business action is to maintain sufficient inventory for high-performing products, continue targeted marketing campaigns, and analyze customer demand to maintain the category's performance.

4. Conclusion

The practical successfully demonstrated the use of **Apache Spark DataFrames for e-commerce sales analysis**. The dataset was loaded and validated, and important business metrics such as revenue, category-wise sales, product quantity, and city-wise revenue were calculated.

The use of operations such as `withColumn()`, `groupBy()`, `sum()`, `avg()`, `orderBy()` and `limit()` helped perform efficient data analysis. Spark provides a scalable approach for processing large e-commerce datasets and generating useful business insights.

The analysis can help businesses identify high-performing categories and products, understand geographical revenue distribution, improve marketing strategies, and make data-driven decisions.

5. Github Link

Github: https://github.com/RudraMaiyariya/D25DIT081_BDA_SHP_2026/tree/main/Practical6